

A Tutorial on Orchestration and Observability of Edge Inference via CI/CD Pipelines

The framework outlined here has been developed by the authors to deploy near real-time applications on heterogeneous edge clusters by addressing resource tuning, secure credentials, and decentralised monitoring. It includes the integration of CI/CD and orchestration best practices for edge environments, sensor-aware scheduling, and a comprehensive observability solution.



setup minimises overhead on non-camera nodes while enabling targeted deployment of camera-dependent pods (see Figure 1).

This work addresses three underexplored challenges in edge inference:

- *ARM64-optimised orchestration:* K3s modifications for Pi-specific resource constraints (GPU memory contention, CPU throttling)
- *Observability under bandwidth constraints:* Metric/lossy compression techniques for low-bandwidth edge monitoring
- *Fault-tolerant CI/CD:* Automated recovery pipelines for edge network instability

Our implementation reveals new insights into tradeoffs between inference latency (Figure 3), resource allocation, and deployment resilience — findings applicable to distributed edge deployments beyond violence detection.

Each pod in the system contains two containers. The Detector container hosts the fine-tuned MobileNetV2+BiLSTM violence-detection model (weights stored on AWS S3). On container startup, it fetches the model from S3 using Kubernetes Secrets (populated via `--extra-args`). The container captures video frames from the attached camera (`/dev/video0`). When violence is detected, the Detector writes a 10-second video clip (5 seconds before

Deploying near real-time applications on distributed edge devices needs to balance computational efficiency and resource constraints with operational reliability. Resource-constrained devices like the Raspberry Pi 4 (ARM64, 4GB RAM) are cost-effective but lack GPU acceleration and have limited compute and memory. In the framework that we have built, multiple Pi 4 worker

nodes (each with an attached camera) connect to a centralised K3s control plane (hosted on an AWS EC2 t2.micro instance). The K3s server (control plane) coordinates container scheduling and maintains cluster state using SQLite (omitting etcd). To ensure only camera-enabled nodes process video, we label Pi nodes with `camera=true` and use a Kubernetes node selector in pod deployments. This lightweight edge